A

Capstone Project Synopsis

On

Pneumonia Disease Detection Using Machine Learning: RespiraCheck

Submitted during 4th semester in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Engineering

by

Vaani (24001003136)

Under supervision of

Dr. Umesh

Department of Computer Engineering
FACULTY OF INFORMATICS & COMPUTING

J.C. Bose University of Science & Technology, YMCA
Faridabad – 121006

Jan-May 2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

Pneumonia is a serious respiratory infection that inflames the air sacs in one or both lungs, and timely diagnosis is critical for effective treatment. Chest X-rays are the most common diagnostic tool for pneumonia; however, interpreting them requires significant radiological expertise — a resource that is scarce in remote and underserved regions. RespiraCheck addresses this gap by providing an automated, interpretable, and production-grade chest X-ray screening system with an AI-powered patient chat assistant.

1.1 BRIEF OF THE PROJECT

RespiraCheck is a full-stack deep learning application for binary classification of chest X-ray images (NORMAL vs. PNEUMONIA). Two backbone architectures — DenseNet121 and EfficientNetB3 — are trained in parallel using a two-phase transfer learning strategy and automatically evaluated on a held-out test set. The model with the higher AUC-ROC score is deployed; in completed testing, DenseNet121 achieved 88.78% accuracy, AUC-ROC of 0.9549, and 92.31% pneumonia recall, exceeding all stated targets.

Grad-CAM++ heatmaps are generated for every prediction, visually highlighting the lung regions that drove the classification. A Google Gemini 2.5 Flash powered chat assistant is integrated via a /chat API endpoint, enabling patients to ask questions about their result and learn about pneumonia in accessible language. The final deliverable is a Flask REST API (/predict, /chat, /health) paired with a React frontend.

1.2 TECHNOLOGIES USED

- Programming Language: Python 3.10+
- Deep Learning: TensorFlow 2.20.0, tf_keras 2.20.0
- Architectures: DenseNet121, EfficientNetB3 (ImageNet weights)
- Explainability: Grad-CAM++ via tf-keras-vis 0.8.7
- AI Chat: Google Gemini 2.5 Flash (google-genai 1.68.0)
- Data / Visualisation: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, OpenCV, Pillow
- Backend / Frontend: Flask 3.1.0, Flask-CORS 5.0.1, React (JavaScript / TypeScript)
- Dev Environment: VS Code, Google Colab, Jupyter Notebook

1.3 OBJECTIVES OF THE PROJECT

- Develop and compare DenseNet121 and EfficientNetB3 for pneumonia classification using a standardised pipeline.
- Implement two-phase transfer learning: frozen-backbone head training followed by selective fine-tuning of the top 30 layers.
- Automate model selection based on AUC-ROC and deploy the best model via Flask REST API and React frontend.
- Integrate Grad-CAM++ so every prediction is accompanied by a visual heatmap overlay.

- Integrate a Gemini 2.5 Flash AI chat assistant for patient education and result explanation.
- Achieve test accuracy > 80%, pneumonia recall > 93%, and AUC-ROC > 0.92.

CHAPTER 2: PROPOSED METHODOLOGY

The pipeline is executed in six structured phases:

1. DATA ACQUISITION AND PREPROCESSING

The Kaggle Chest X-Ray Images (Pneumonia) dataset (5,856 images; 1,583 NORMAL, 4,273 PNEUMONIA) is used. The default 16-image-per-class validation split is discarded; train/ and val/ are merged and re-split 80/20 with stratification. Images are resized to 224×224 pixels, normalised to [0,1], and augmented with random rotations (±20°), horizontal flips, zoom (10%), and shifts (10%). Class weights are computed via sklearn to address the 1:2.7 imbalance.

2. MODEL DEVELOPMENT

Both DenseNet121 and EfficientNetB3 are loaded with ImageNet weights and an identical classification head is appended: GlobalAveragePooling2D → Dense(512, ReLU) → Dropout(0.5) → Dense(1, Sigmoid). This shared head ensures any performance difference is attributable solely to the backbone architecture.

3. TRAINING AND FINE-TUNING

Phase 1: The backbone is fully frozen; the classification head is trained for up to 10 epochs (Adam, LR=1e-4, binary cross-entropy). Phase 2: The last 30 layers of the backbone are unfrozen (Batch Normalisation layers remain frozen) and fine-tuned at LR=1e-5. All phases use EarlyStopping (patience=5), ModelCheckpoint (monitoring val_auc), and ReduceLROnPlateau (factor=0.5, patience=3) callbacks.

4. EVALUATION AND AUTOMATIC MODEL SELECTION

Both Phase 2 models are evaluated on the test set using Accuracy, Recall, Precision, F1-Score, AUC-ROC, Average Precision, and Confusion Matrix. ROC and PR curves for both models are plotted on the same axes for comparison. The model with the higher AUC-ROC is automatically written to best_model_path.txt, which the Flask API reads at startup. DenseNet121 was selected: Accuracy 88.78%, AUC-ROC 0.9549, Pneumonia Recall 92.31%, AP 0.9716, F1 91.14%.

5. EXPLAINABILITY WITH GRAD-CAM++

Grad-CAM++ is applied via tf-keras-vis to the final convolutional layer of the winning model, generating a colour heatmap overlay (blue → red: low → high activation) on the original X-ray. The heatmap is returned as a Base64-encoded PNG by the /predict endpoint alongside the prediction label and confidence score.

6. DEPLOYMENT AND AI CHAT INTEGRATION

The Flask REST API exposes: /predict (image upload → prediction + confidence + Grad-CAM heatmap), /chat (Gemini 2.5 Flash multi-turn assistant that explains results, educates on pneumonia, and guides next steps while directing users to consult a doctor), and /health (liveness probe). The React frontend provides the upload interface and chat window. API keys are managed via python-dotenv.

CHAPTER 3: TOOLS AND TECHNOLOGIES TO BE USED

The following table summarises all tools, frameworks, and libraries used in this project:

Category	Technology / Library
Programming Language	Python 3.10+
Deep Learning Framework	TensorFlow 2.20.0, tf_keras 2.20.0
Pre-trained Architectures	DenseNet121, EfficientNetB3 (ImageNet weights)
Explainability	Grad-CAM++ via tf-keras-vis 0.8.7
AI Chat Assistant	Google Gemini 2.5 Flash (google-genai 1.68.0)
Data Handling	NumPy 2.2.4, Pandas 2.2.3, Scikit-learn 1.6.1
Image Processing	OpenCV 4.11.0, Pillow 11.2.1
Visualization	Matplotlib 3.10.1, Seaborn 0.13.2
Backend / Config	Flask 3.1.0, Flask-CORS 5.0.1, python-dotenv 1.2.2
Frontend	React (JavaScript / TypeScript)
Dataset	Kaggle: Chest X-Ray Images (Pneumonia) — 5,856 images
Dev Environment	VS Code, Google Colab, Jupyter Notebook

CHAPTER 4: APPLICATIONS OF THE PROJECT

4.1 APPLICATIONS

- Clinical Decision Support: Provides a fast second opinion for radiologists, with Grad-CAM++ heatmaps pinpointing suspicious lung regions and helping prioritise critical cases.
- Telemedicine and Remote Diagnostics: Deployable in mobile clinics or rural areas lacking specialist radiologists; only a browser and internet connection are required.
- Patient Education via AI Chat: The Gemini 2.5 Flash assistant lets patients ask questions about their diagnosis in plain language, guiding them on next steps within the same interface.
- Medical Education and Training: Students and interns can compare their own Xray readings with AI-identified activation regions to reinforce radiographic pattern recognition skills.
- Scalable Public Health Screening: Serves as a rapid automated pre-screening layer in high-volume workflows, flagging probable positive cases for priority expert review.

4.2 SCOPE AND LIMITATIONS

Scope: RespiraCheck is a proof-of-concept for binary classification (Pneumonia vs. Normal) on the Kaggle dataset, demonstrating a rigorous, interpretable pipeline that significantly improves on the Semester 3 ResNet50 prototype (74.84% accuracy).

- The system is a pre-diagnostic screening tool and must not replace a qualified radiologist; all predictions require medical verification.
- Performance may degrade on X-rays from imaging equipment not represented in the training data.
- Binary classification only — cannot diagnose tuberculosis, COVID-19, pleural effusion, or other thoracic conditions.
- The /chat endpoint requires an active internet connection and a valid Gemini API key.

CHAPTER 5: CONCLUSION

RespiraCheck represents a substantial advancement over the Semester 3 ResNet50 prototype, evolving into a rigorous, production-oriented medical AI system. By training and comparing DenseNet121 and EfficientNetB3 with two-phase transfer learning, automatically selecting the best model, handling class imbalance, incorporating GradCAM++ explainability, and integrating a Gemini 2.5 Flash AI chat assistant, the project addresses the key shortcomings of typical AI medical imaging tools: opacity, singlemodel bias, and lack of patient-facing communication.

The completed evaluation confirms DenseNet121 as the winner with 88.78% test accuracy, AUC-ROC of 0.9549, and 92.31% pneumonia recall — surpassing all stated performance targets. The Flask REST API and React frontend make the system accessible to non-technical medical personnel. While limited to binary pneumonia classification on the Kaggle dataset, the architecture is designed for extensibility to multi-class thoracic disease classification in future iterations.

Note: RespiraCheck is a pre-diagnostic decision support tool and is not intended to replace a qualified radiologist. All predictions must be verified by a medical professional before any clinical action is taken.

CHAPTER 6: FUTURE WORK

- Multi-Class Classification: Extend the model to distinguish bacterial pneumonia, viral pneumonia, COVID-19, tuberculosis, and pleural effusion using the NIH ChestX-ray14 dataset.
- Pneumonia Localisation: Train a YOLOv8 detection model on RSNA bounding box annotations to precisely localise infected lung regions beyond heatmap-level explainability.

- Multimodal AI Chat: Upgrade the Gemini assistant to directly analyse the GradCAM++ heatmap image for image-aware, context-specific explanations.
- Clinical Validation and Regulatory Pathway: Partner with hospitals for multicentre validation and explore CE/FDA regulatory pathways for clinical deployment.
- Mobile Application: Build a lightweight mobile version using TensorFlow Lite model quantisation for use on low-resource devices in remote health centres.
- Federated Learning: Train collaboratively across hospital networks without centralising patient records, addressing HIPAA/GDPR privacy requirements.

REFERENCES

1. TensorFlow Documentation (2024). https://www.tensorflow.org/
2. Flask Documentation (2024). https://flask.palletsprojects.com/
3. Kaggle Chest X-Ray Images (Pneumonia) Dataset (2018). https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia
4. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K.Q. (2017). Densely Connected Convolutional Networks. CVPR.
5. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML.
6. Rajpurkar, P. et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays. Stanford ML Group.
7. Selvaraju, R.R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. ICCV.
8. Chattopadhyay, A. et al. (2018). Grad-CAM++: Generalized Gradient-Based Visual Explanations. WACV.
9. RSNA Pneumonia Detection Challenge (2018). https://www.kaggle.com/c/rsnapneumonia-detection-challenge
10. World Health Organization (2022). Pneumonia Fact Sheet. https://www.who.int/news-room/fact-sheets/detail/pneumonia
11. Scikit-learn Documentation. https://scikit-learn.org/stable/
12. tf-keras-vis Library. https://github.com/keisen/tf-keras-vis
13. Google Generative AI SDK (google-genai). https://ai.google.dev/